ECS 170

Artificial Intelligence

Fall 2026

*Course Project: Stage **2** Report*

Team Information

Enter Your Team Name Here **(delete the extra rows if your team has less than 5 students)**		
Student 1: Name	Student 1: ID	Student 1: Email
Student 2: Name	Student 2: ID	Student 2: Email
Student 3: Name	Student 3: ID	Student 3: Email
Student 4: Name	Student 4: ID	Student 4: Email
Student 5: Name	Student 5: ID	Student 5: Email

Section 1: Task Description

In this stage we build a multi-layer perceptron (MLP) with PyTorch and use it for multi-class classification. Every instance is a 28×28 grayscale image of a hand-written digit, flattened into 784 integer pixel features, and the task is to predict which of the 10 digits (0–9) the image shows. The model is trained on the provided training set, applied to the separate testing set, and evaluated with accuracy, precision, recall and F1. We then change the model depth, layer width, activation function, optimizer, learning rate, loss function and dropout rate one at a time to study how each setting affects the learning performance.

Section 2: Model Description

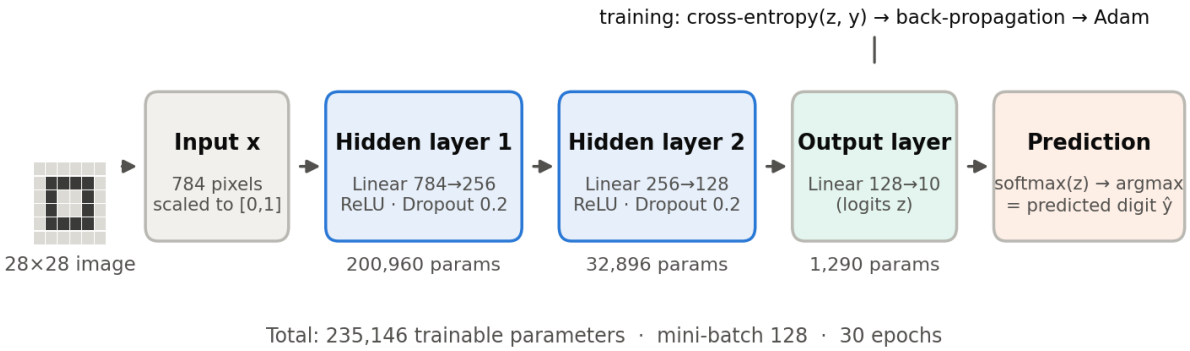


Figure 1. Architecture of the final MLP (784 → 256 → 128 → 10).

The model is a fully connected feed-forward network with two hidden layers (Figure 1). The input $x \in \mathbb{R}^{784}$ passes through **Linear(784→256)** → **ReLU** → **Dropout(0.2)**, then **Linear(256→128)** → **ReLU** → **Dropout(0.2)**, and finally **Linear(128→10)**, which outputs one score (logit) z_k per digit. Each hidden layer computes $h = \mathrm{ReLU}(Wh' + b)$, where h' is the previous layer's output. ReLU avoids the saturation (vanishing

gradients) of sigmoid and tanh units, and dropout randomly zeroes 20% of the hidden units at every training step to reduce overfitting; it is switched off at test time.

The output layer returns raw logits. The training loss is the cross-entropy between softmax(z) and the one-hot true label, $L = -\log[\exp(z_y) / \Sigma_k \exp(z_k)]$ with y the true digit, which PyTorch's `nn.CrossEntropyLoss` computes directly from the logits. This differs from the Stage 1 template, which placed a Softmax layer *before* `CrossEntropyLoss`: since the loss already applies log-softmax, the probabilities get squashed a second time, the loss can never drop below $\log(1 + 9/e) \approx 1.46$, and the gradients shrink (Section 3.7 shows this costs about 0.5 accuracy points). The predicted class is the digit with the largest logit. All weights are learned by back-propagation with mini-batch gradient descent using the Adam optimizer. The model has 235,146 trainable parameters.

Section 3: Experiment Settings

3.1 Dataset Description

We use the Stage 2 dataset provided by the instructor: two CSV files, **train.csv** (60,000 instances) and **test.csv** (10,000 instances). The training/testing partition is given, so no random split or cross validation is used. Each line contains 785 comma-separated integers: the class label (0–9) followed by the 784 pixel intensities (0–255) of a 28×28 digit image, the same format as the MNIST benchmark. The ten classes are roughly balanced (5,421 to 6,742 training instances and 892 to 1,135 testing instances per digit). The only preprocessing is dividing every feature by 255 so the inputs lie in [0, 1].

The testing set is never used to update the weights; it is only evaluated after every epoch to draw the learning curves. Since no separate validation set is provided, the comparisons in the ablation study (Section 3.7) are also measured on the testing set.

3.2 Detailed Experimental Setups

Only one dataset is used in this stage, so there is a single setting (Table 1). Every run in the ablation study uses the same setup except for the one item being changed, and trains for 20 epochs so all variants get the same budget.

Setting	Value
Architecture (depth)	2 hidden layers: 784 → 256 → 128 → 10 (fully connected)
Activation / regularization	ReLU after each hidden layer, Dropout p = 0.2 after each hidden layer
Output / loss	10 raw logits; nn.CrossEntropyLoss (log-softmax + negative log-likelihood)
Optimizer	Adam, learning rate 0.001, betas (0.9, 0.999), no weight decay
Batch size / epochs	Mini-batch 128, shuffled every epoch (469 updates/epoch); 30 epochs
Initialization	PyTorch default for nn.Linear: weights and biases ~ U(−1/√fan_in, 1/√fan_in)
Input preprocessing	Pixel values divided by 255
Random seed	numpy and torch seed = 2
Environment	Python 3.10, PyTorch 2.5.1, scikit-learn 1.7.2; CPU (2 cores), ~35 s of training

Table 1. Setup of the final model.

3.3 Evaluation Metrics

In the experiment, we will use Accuracy, Precision, Recall, F1 score as our evaluation metrics. Accuracy is the fraction of test instances whose predicted label equals the true label. For each class c, precision = TP/(TP+FP) (how many images predicted as c really are c), recall = TP/(TP+FN) (how many images of c are found), and F1 = 2·P·R/(P+R), their harmonic mean.

Because this is a 10-class problem, the binary versions cannot be used; we report three averages computed with `sklearn.metrics`: **macro** (unweighted mean over the 10 classes, so every digit counts equally), **weighted** (mean weighted by the number of test instances of each class), and **micro** (TP, FP and FN pooled over all classes; for single-label multi-class data, micro precision = micro recall = micro F1 = accuracy).

3.4 Source Code

Code link: [paste the GitHub / Google Drive link to the stage2submittion folder here]

The code follows the course template (abstract classes in `local_code/base_class/`). The Stage 2 files are:

- `local_code/stage_2_code/Dataset_Loader.py` – reads a CSV file into features X and labels y, scales pixels to [0, 1].
- `Method_MLP.py` – configurable MLP (depth, width, activation, dropout, optimizer, learning rate, loss) with the mini-batch training loop and learning-curve history.
- `Evaluate_Metrics.py` – accuracy and macro / weighted / micro precision, recall, F1; `Setting_Train_Test.py` – runs the pre-partitioned train/test experiment; `Result_Saver.py`, `Result_Loader.py`, `Plotter.py`.
- `script/stage_2_script/script_mlp.py` – trains and evaluates the final model; `script_ablation.py` – runs the 24 ablation experiments and writes the results table and plots.

3.5 Training Convergence Plot

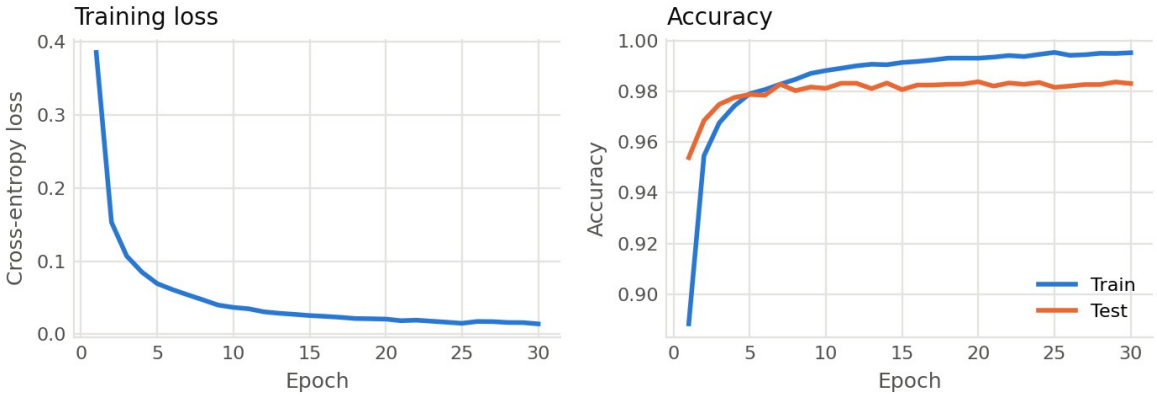


Figure 2. Training convergence of the final model: training loss (left) and train/test accuracy (right) per epoch.

The training loss falls quickly from 0.386 after the first epoch to 0.069 after epoch 5, then keeps decreasing slowly to 0.014 at epoch 30, so the gradient-descent training converges smoothly. Test accuracy rises to about 98% within 5 epochs and stays between 98.06% and 98.37% from epoch 10 on, while training accuracy keeps climbing to 99.5%. The later epochs therefore mostly fit the training set; dropout keeps the final train/test gap small (about 1.2 points; training accuracy is measured with dropout active).

3.6 Model Performance

Metric	Macro	Weighted	Micro
Precision	0.9831	0.9830	0.9830
Recall	0.9828	0.9830	0.9830
F1 score	0.9829	0.9830	0.9830
Accuracy	0.9830		

Table 2. Performance of the final model on the 10,000-instance testing set.

The final model classifies 9,830 of the 10,000 test images correctly (**accuracy 98.30%**, macro F1 0.9829). Because the classes are nearly balanced, the macro, weighted and micro scores are almost identical. Per-class F1 ranges from 0.975 (digit 8) and 0.976 (digit 9) up to 0.993 (digit 0) and 0.991 (digit 1). The most frequent mistakes are between visually similar digits: 9 predicted as 4 (12 times), 3 as 7 (8), 7 as 9 (7) and 4 as 6 (7).

3.7 Ablation Studies

Starting from a reference model (784-256-128-10, ReLU, dropout 0.2, Adam with learning rate 0.001, cross-entropy on logits), we changed one setting at a time and trained each variant for 20 epochs with the same seed. Table 3 lists the test scores (precision, recall and F1 are weighted averages) and Figure 3 shows the test-accuracy curves of three of the groups.

Change from reference	Setting	Accuracy	Precision	Recall	F1	Params
Reference	**256-128, ReLU, Adam 0.001, CE, dropout 0.2**	**0.9837**	**0.9837**	**0.9837**	**0.9837**	**235,146**
Depth	0 hidden layers (784-10)	0.9284	0.9282	0.9284	0.9282	7,850
Depth	1 hidden layer (256)	0.9827	0.9827	0.9827	0.9827	203,530
Depth	3 hidden layers (256-128-64)	0.9835	0.9835	0.9835	0.9835	242,762
Depth	4 hidden layers (256-128-64-32)	0.9796	0.9797	0.9796	0.9796	244,522
Width	64-32	0.9748	0.9748	0.9748	0.9748	52,650
Width	128-64	0.9803	0.9803	0.9803	0.9803	109,386
Width	512-256	0.9825	0.9825	0.9825	0.9825	535,818
Width	1024-512	0.9826	0.9827	0.9826	0.9826	1,333,770
Activation	Sigmoid	0.9802	0.9803	0.9802	0.9802	235,146
Activation	Tanh	0.9800	0.9800	0.9800	0.9800	235,146
Optimizer	SGD + momentum 0.9 (lr 0.01)	0.9814	0.9814	0.9814	0.9814	235,146
Optimizer	Adagrad (lr 0.01)	0.9814	0.9814	0.9814	0.9814	235,146
Learning rate	Adam lr 0.01	0.9746	0.9748	0.9746	0.9746	235,146
Learning rate	Adam lr 0.0001	0.9759	0.9759	0.9759	0.9759	235,146
Loss function	Softmax layer + cross-entropy (template)	0.9789	0.9789	0.9789	0.9789	235,146
Loss function	MSE on softmax vs. one-hot	0.9829	0.9829	0.9829	0.9829	235,146
Dropout	No dropout	0.9784	0.9786	0.9784	0.9784	235,146
Dropout	Dropout 0.5	0.9817	0.9817	0.9817	0.9817	235,146

Table 3. Ablation study (20 epochs each; one setting changed from the reference in the first row).

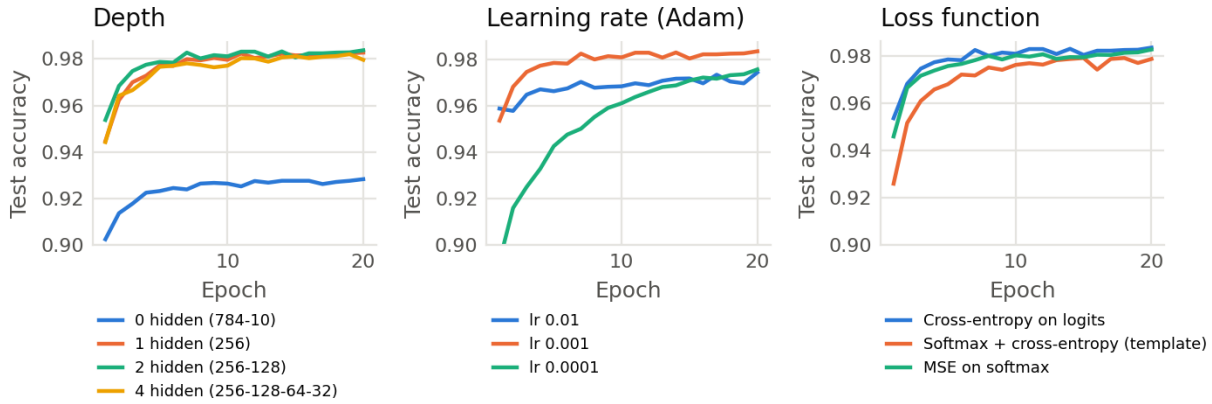


Figure 3. Test accuracy per epoch for the depth, learning-rate and loss-function ablations.

- **Depth.** Adding a hidden layer matters most: the model without hidden layers (multinomial logistic regression) reaches only 92.84%, 5.5 points below the reference, because it can only draw linear decision boundaries. One hidden layer already gives 98.27%, two give the best result (98.37%), and a third adds nothing (98.35%). With four layers the narrow 32-unit layer becomes a bottleneck and accuracy drops to 97.96%.
- **Width.** Accuracy grows from 97.48% (64-32) to 98.37% (256-128). Wider layers (512-256, 1024-512) do not help (98.25–98.26%) even with 2.3× and 5.7× more parameters and 2–4× longer training: they fit the training set better (99.5% vs 99.3%) but do not generalize better.

- **Activation and optimizer.** ReLU beats sigmoid (98.02%) and tanh (98.00%); the saturating functions also learn more slowly in the first epochs (91.6% vs 95.4% test accuracy after epoch 1 for sigmoid vs ReLU). Adam (98.37%) is better than SGD with momentum and Adagrad (both 98.14%), and SGD starts much more slowly.
- **Learning rate.** It is the most sensitive hyper-parameter. With 0.01 the Adam steps are too large and training is noisy (97.46%); with 0.0001 training is too slow and is still improving after 20 epochs (97.59%). 0.001 is best.
- **Loss function.** The template's Softmax layer followed by CrossEntropyLoss reaches only 97.89% (−0.48 points) and its training loss stalls near 1.48, as explained in Section 2. Cross-entropy on raw logits is best; MSE between the softmax output and the one-hot label is close (98.29%) but learns more slowly early on.
- **Dropout.** Without dropout the model overfits (99.90% training vs 97.84% test accuracy). Dropout 0.2 gives the best test accuracy; 0.5 regularizes too strongly (98.17%).

The final model uses the best setting from every group, i.e. the reference configuration, trained for 30 epochs instead of 20. It scores 98.30% (Table 2) versus 98.37% at 20 epochs; the difference is within the epoch-to-epoch fluctuation of the test accuracy (Figure 2), confirming that the model has converged by about epoch 20.